

HDOC Day-25

Decreasing Patterns Using Nested Loops and switch-case in C

Problem: Create a menu-driven C program using switch-case. Each decreasing pattern must be implemented in a separate parameterless function with no return value. Use nested loops and do not use arrays.

1. Algorithm

1. Start.
2. Display the menu: 1) Decreasing right-angled triangle, 2) Decreasing natural-number pattern, 3) Decreasing repeated-number pattern, 4) Reverse-number triangle, 5) Exit.
3. Read the user choice.
4. Use a switch statement. For choices 1 to 4, call the corresponding parameterless function.
5. Inside the selected function, read the number of rows n. If $n \leq 0$, display an error message and return to the menu.
6. Choice 1: for i from n down to 1, print i asterisks using an inner loop.
7. Choice 2: for i from n down to 1, print natural numbers 1 through i.
8. Choice 3: for i from n down to 1, print the value i exactly i times.
9. Choice 4: for i from n down to 1, print numbers i down to 1.
10. For an invalid menu choice, display an appropriate error message.
11. Repeat the menu until the user selects 5. Stop.

2. Test-Case Table

TC	Choice	Input n	Purpose	Expected output / behavior
1	1	4	Valid	Rows contain 4, 3, 2, 1 asterisks respectively.
2	2	4	Valid	Prints: 1 2 3 4; 1 2 3; 1 2; 1.
3	3	4	Valid	Prints: 4 4 4 4; 3 3 3; 2 2; 1.
4	4	4	Valid	Prints: 4 3 2 1; 3 2 1; 2 1; 1.
5	1-4	1	Boundary	Prints exactly one row containing the appropriate single symbol/number.
6	1-4	0	Invalid	Displays: Number of rows must be positive.
7	9	-	Invalid menu	Displays: Invalid choice. Please enter 1 to 5.
8	5	-	Exit	Displays exit message and terminates the program.

3. C Program

```
#include <stdio.h>
```

```
void decreasingStarTriangle(void);
```

```

void decreasingNaturalPattern(void);
void decreasingRepeatedPattern(void);
void reverseNumberTriangle(void);

int main(void)
{
    int choice;

    do
    {
        printf("\n--- DECREASING PATTERNS MENU ---\n");
        printf("1. Decreasing right-angled triangle\n");
        printf("2. Decreasing natural-number pattern\n");
        printf("3. Decreasing repeated-number pattern\n");
        printf("4. Reverse-number triangle\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");

        if (scanf("%d", &choice) != 1)
        {
            printf("Invalid input. Please enter a number from 1 to 5.\n");
            while (getchar() != '\n') { }
            continue;
        }

        switch (choice)
        {
            case 1:
                decreasingStarTriangle();
                break;
            case 2:
                decreasingNaturalPattern();
                break;
            case 3:
                decreasingRepeatedPattern();
                break;
            case 4:
                reverseNumberTriangle();
                break;
            case 5:
                printf("Exiting program.\n");
                break;
            default:
                printf("Invalid choice. Please enter 1 to 5.\n");
        }
    } while (choice != 5);

    return 0;
}

void decreasingStarTriangle(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }

    for (i = n; i >= 1; i--)
    {
        for (j = 1; j <= i; j++)
            printf("* ");
        printf("\n");
    }
}

void decreasingNaturalPattern(void)
{

```

```

    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }

    for (i = n; i >= 1; i--)
    {
        for (j = 1; j <= i; j++)
            printf("%d ", j);
        printf("\n");
    }
}

void decreasingRepeatedPattern(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }

    for (i = n; i >= 1; i--)
    {
        for (j = 1; j <= i; j++)
            printf("%d ", i);
        printf("\n");
    }
}

void reverseNumberTriangle(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }

    for (i = n; i >= 1; i--)
    {
        for (j = i; j >= 1; j--)
            printf("%d ", j);
        printf("\n");
    }
}

```

4. Sample Outputs (n = 4)

Choice 1 - Decreasing right-angled triangle

```

* * * *
* * *
* *
*

```

Choice 2 - Decreasing natural-number pattern

```

1 2 3 4

```

```
1 2 3
1 2
1
```

Choice 3 - Decreasing repeated-number pattern

```
4 4 4 4
3 3 3
2 2
1
```

Choice 4 - Reverse-number triangle

```
4 3 2 1
3 2 1
2 1
1
```

5. Screenshot

Screenshot : Source code / successful compilation / output

```
#include <stdio.h>
void decreasingStarTriangle(void);
void decreasingNaturalPattern(void);
void decreasingRepeatedPattern(void);
void reverseNumberTriangle(void);
int main(void)
{
    int choice;
    do
    {
        printf("\n— DECREASING PATTERNS MENU —\n");
        printf("1. Decreasing right-angled triangle\n");
        printf("2. Decreasing natural-number pattern\n");
        printf("3. Decreasing repeated-number pattern\n");
        printf("4. Reverse-number triangle\n");
        printf("5. Exit\n");
        printf("Enter your choice: ");
        if (scanf("%d", &choice) != 1)
        {
            printf("Invalid input. Please enter a number from 1 to 5.\n");
            while (getchar() != '\n') { }
            continue;
        }
        switch (choice)
        {
            case 1:
                decreasingStarTriangle();
                break;
            case 2:
                decreasingNaturalPattern();
                break;
            case 3:
                decreasingRepeatedPattern();
                break;
            case 4:
                reverseNumberTriangle();
                break;
            case 5:
                exit(0);
                break;
            default:
                printf("Invalid choice. Please enter a number from 1 to 5.\n");
                continue;
        }
    } while (choice != 5);
}
```

```

break;
case 3:
    decreasingRepeatedPattern();
    break;
case 4:
    reverseNumberTriangle();
    break;
case 5:
    printf("Exiting program.\n");
    break;
default:
    printf("Invalid choice. Please enter 1 to 5.\n");
}
} while (choice != 5);
return 0;
}

void decreasingStarTriangle(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    if (n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }
    for (i = n; i >= 1; i--)
    {
        for (j = 1; j <= i; j++)
            printf("* ");
        printf("\n");
    }
}

```

```

}
}
}

void decreasingNaturalPattern(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    if (n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }
    for (i = n; i >= 1; i--)
    {
        for (j = 1; j <= i; j++)
            printf("%d ", j);
        printf("\n");
    }
}

void decreasingRepeatedPattern(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    if (n <= 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }
    for (i = n; i >= 1; i--)
    {
        for (j = 1; j <= i; j++)

```

```

    for (j = 1; j ≤ i; j++)
        printf("%d ", i);
    printf("\n");
}
}

void reverseNumberTriangle(void)
{
    int n, i, j;
    printf("Enter number of rows: ");
    scanf("%d", &n);
    if (n ≤ 0)
    {
        printf("Number of rows must be positive.\n");
        return;
    }
    for (i = n; i ≥ 1; i--)
    {
        for (j = i; j ≥ 1; j--)
            printf("%d ", j);
        printf("\n");
    }
}

```

```

(kali㉿kali)-[~/Desktop]
$ vi hdoc25.c

(kali㉿kali)-[~/Desktop]
$ gcc hdoc25.c -o hdoc25 -lm

```

```

(kali㉿kali)-[~/Desktop]
$ ./hdoc25

— DECREASING PATTERNS MENU —
1. Decreasing right-angled triangle
2. Decreasing natural-number pattern
3. Decreasing repeated-number pattern
4. Reverse-number triangle
5. Exit
Enter your choice: 1
Enter number of rows: 4
* * * *
* * *
* *
*

```

```

5. EXIT
Enter your choice: 2
Enter number of rows: 4
1 2 3 4
1 2 3
1 2
1

```

```

Enter your choice: 3
Enter number of rows: 4
4 4 4 4
3 3 3
2 2
1

```

```
5. Exit
Enter your choice: 4
Enter number of rows: 4
4 3 2 1
3 2 1
2 1
1
```

```
5. Exit
Enter your choice: 1
Enter number of rows: 1
*
```

```
5. Exit
Enter your choice: 2
Enter number of rows: 0
Number of rows must be positive.
```

```
5. Exit
Enter your choice: 9
Invalid choice. Please enter 1 to 5.
```

```
5. Exit
Enter your choice: 5
Exiting program.
```